Algorytmy geometryczne - sprawozdanie 4

Przecinanie odcinków

Adrianna Grobel
Data: 01.12.2025

1. Dane techniczne

- System operacyjny - macOS
- Procesor - Apple M3
- Pamięć RAM - 16 GB
- Środowisko - Jupyter Notebook
- Język - Python 3.13.5
- Wykorzystane biblioteki - Numpy, Pandas, Matplotlib, SortedContainers

2. Cel ćwiczenia

Celem ćwiczenia było zapoznanie się z problemem wykrywania przecięć odcinków na płaszczyźnie oraz praktyczna implementacja algorytmu zamiatania (sweep line). W ramach zadania należało odpowiednio zbudować strukturę zdarzeń i strukturę stanu, przeanalizować działanie algorytmu pod kątem wykrywania istnienia co najmniej jednego przecięcia oraz znajdowania wszystkich punktów przecięć, a także zwizualizować kolejne etapy działania algorytmu

3. Opis i implementacja algorytmu

Implementując algorytm zakładamy, że zbiór odcinków spełnia następujące założenia:

- Żaden odcinek nie jest pionowy,
- Dwa odcinki mogą przecinać się w co najwyżej jednym punkcie,
- Żadne trzy odcinki nie przecinają się w jednym punkcie,
- Żadne dwa odcinki nie mają końców o tej samej współrzędnej x.

3.1. Algorytm zamiatania

Zaimplementowany algorytm zamiatania służy do identyfikacji przecięć odcinków na płaszczyźnie. Fundamentem jego działania jest hiperpłaszczyzna, w $\mathbb{R}^2$ jest to pionowa prosta, zwana miotłą. Prosta przemieszcza się wzdłuż osi OX zatrzymując się w punktach określanych mianem zdarzeń.

Mechanizm ten opiera się na współpracy dwóch struktur danych:

- **Struktura stanu T** - posortowany względem współrzędnej y zbiór odcinków aktualnie aktywnych, czyli przecinających miotłę.
- **Struktura zdarzeń Q** - zbiór uporządkowanych rosnąco, względem współrzędnej x, punktów początkowych i końcowych odcinków, a także wykrytych punktów przecięć.

Algorytm rozpoczyna działanie od inicjalizacji pustej struktury stanu T oraz wypełnienia struktury zdarzeń Q punktami końcowymi wszystkich odcinków. Główna

pętla algorytmu iteracyjnie pobiera zdarzenie p o najmniejszej współrzędnej x, co odpowiada przesunięciu miotły w prawo. W każdym kroku następuje aktualizacja struktury stanu T, po czym sprawdzane są nowe relacje sąsiedztwa w T pod kątem kolizji. Następnie aktualizowana jest kolejka Q, co polega na usunięciu obsłużonego zdarzenia oraz ewentualnym dodaniu nowych zdarzeń wynikających ze znalezionych przecięć. Przetworzony obszar na lewo od miotły uznaje się za w pełni przeanalizowany. Algorytm kończy działanie po przetworzeniu wszystkich punktów ze struktury zdarzeń.

3.2. Struktura zdarzeń Q

W strukturze zdarzeń przechowywane są punkty determinujące zmiany w strukturze stanu, uporządkowane rosnąco po współrzędnej x. Zbiór ten składa się z trzech typów elementów: punktów początkowych i końcowych odcinków oraz dynamicznie wykrywanych punktów przecięć.

W implementacji algorytmu badającego istnienie przecięć, jako strukturę zdarzeń (Q) wykorzystano standardową listę języka Python. Z uwagi na fakt, że procedura przerywa działanie natychmiast po wykryciu pierwszej kolizji i nie wymaga dynamicznego dodawania nowych zdarzeń, rozwiązanie to jest optymalne. Wstępne posortowanie końców odcinków w kolejności malejącej pozwala na pobieranie zdarzenia o najmniejszej odciętej w amortyzowanym czasie $O(1)$. W wariancie algorytmu wyznaczającym zbiór wszystkich przecięć, konieczne było zastąpienie listy kopcem binarnym `heapq`. Wynika to z potrzeby dynamicznego wprowadzania do kolejki nowych zdarzeń (wykrytych punktów przecięć), co w przypadku listy wiązałoby się ze złożonością liniową. Kopiec umożliwia zarówno pobieranie minimum, jak i wstawianie nowych punktów w czasie $O(\log n)$.

3.3. Struktura stanu T

Struktura stanu to dynamicznie aktualizowana kolekcja odcinków znajdujących się na bieżącej linii miotły, posortowana według wartości y. Dzięki zachowaniu tego porządku, możliwe jest efektywne wyszukiwanie, wstawianie, usuwanie odcinków oraz określanie relacji sąsiedztwa między nimi. Algorytm nie musi sprawdzać każdej pary odcinków, testy na przecięcie ograniczają się wyłącznie do par, które w strukturze stanu ze sobą sąsiadują, co drastycznie redukuje liczbę wykonanych operacji geometrycznych. Aktualizacja struktury T zależy od typu przetwarzanego zdarzenia:

- **Zdarzenie początku odcinka:** nowy odcinek zostaje wprowadzony do struktury T i zajmuję pozycję wynikającą z jego wartości y w aktualnym punkcie linii miotły. Następnie algorytm weryfikuje, czy nowo dodany element przecina się ze swoim bezpośrednim poprzednikiem lub następnikiem w posortowanym zbiorze.
- **Zdarzenie końca odcinka:** odcinek jest usuwany ze struktury. Usunięcie elementu sprawia, że jego dotychczasowi sąsiedzi stają się nową parą sąsiadującą. Algorytm sprawdza, czy odcinki te posiadają punkt przecięcia na prawo od miotły.
- **Zdarzenie przecięcia:** W punkcie przecięcia następuje inwersja kolejności dwóch kolidujących odcinków. W implementacji zrealizowano to poprzez tymczasowe

usunięcie obu odcinków, przesunięcie punktu odniesienia o minimalną wartość (epsilon) i ponowne ich wstawienie. Po zamianie miejscami, sprawdzane są nowe relacje sąsiedztwa dla obu odcinków z ich nowymi zewnętrznymi sąsiadami.

Wykrycie kolizji w którymkolwiek z powyższych kroków skutkuje dodaniem nowego zdarzenia do kolejki zdarzeń Q. Aby uniknąć wielokrotnego dodawania tych samych punktów przecięć do Q, zastosowano dodatkowy zbiór przechowujący informacje o sprawdzonych już parach odcinków.

Weryfikacja występowania punktów przecięć między parą odcinków jest realizowana w sposób analityczny, w oparciu o równania kierunkowe prostych. Algorytm zaczyna od sprawdzenia równoległości poprzez porównanie współczynników kierunkowych. Jeśli są one równe z dokładnością do małego błędu numerycznego, to uznajemy że odcinki się nie przecinają. Dla prostych nierównoległych wyznaczana jest współrzędna x punktu przecięcia, obliczana na podstawie układu równań liniowych. Punkt jest akceptowany tylko wtedy, gdy jego współrzędna x leży w przedziale wyznaczonym przez początki i końce obu analizowanych odcinków.

Od strony technicznej, strukturę stanu oparto na klasie `SortedSet` z biblioteki `SortedContainers`. Zapewnia ona logarytmiczną złożoność czasową $O(\log n)$ dla operacji wstawiania, usuwania i wyszukiwania. Klucz sortowania jest wyznaczany dynamicznie, co pozwala utrzymać właściwy porządek w zbiorze podczas ruchu miotły. Klasa reprezentująca odcinek wykorzystuje statyczną zmienną x, współrzędną linii miotły aktualizowaną w każdym kroku algorytmu, do obliczenia bieżącej wartości y ze wzoru $y = ax + b$. W procesie porównywania dwóch odcinków priorytet ma wyliczona współrzędna y. Jeżeli jednak wartości te są identyczne (w granicach błędu numerycznego), o kolejności decyduje mniejszy współczynnik kierunkowy a, a w ostateczności – unikalny indeks odcinka.

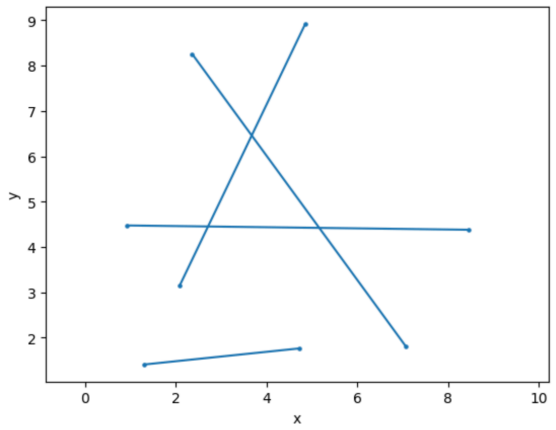
4. Wyniki algorytmu

4.1. Zestawy testowe

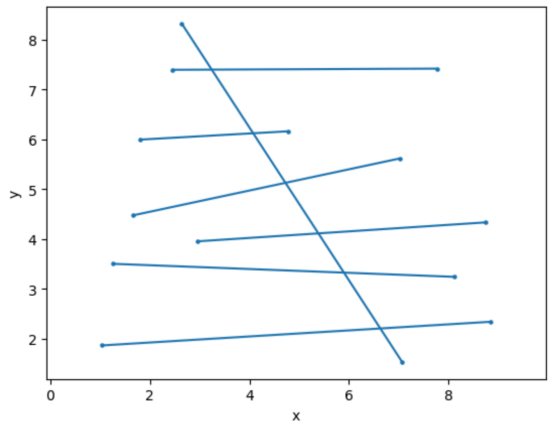
Do testów wykorzystano zarówno dane wygenerowane losowo za pomocą biblioteki `NumPy` z zadanego zakresu współrzędnych, jak i zestawy odcinków wprowadzone interaktywnie poprzez aplikacje graficzną.

Zestaw A składa się z czterech długich odcinków o nielicznych przecięciach. Zestaw B przedstawia sytuacje gdzie wiele punktów przecięcia znajduję się na jednej prostej. Zestaw C ma na celu sprawdzić zachowanie algorytmu, gdy pojedynczy punkt przecięcia będzie wykrywany kilkukrotnie. Zestaw D obejmuje nieprzecinające się odcinki natomiast Zestaw E stanowi gęstą sieć dwudziestu losowo wygenerowanych odcinków z współrzędnymi punktów końcowych w zakresie od 0 do 1000.

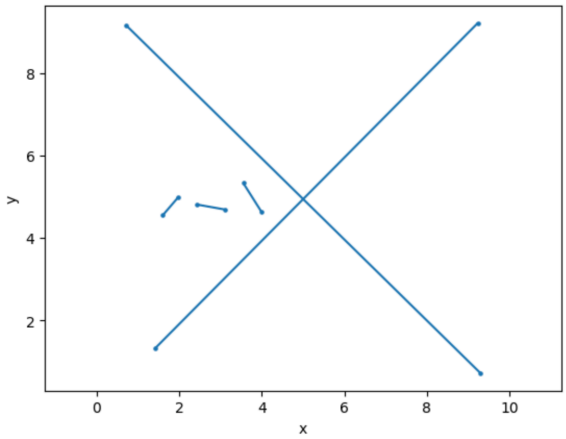
Wygenerowane zbiory zostały przedstawione na Rysunkach 1-5. Zestawy testowe zostały dobrane tak, by sprawdzić poprawność działania algorytmu na na różnorodnych danych.



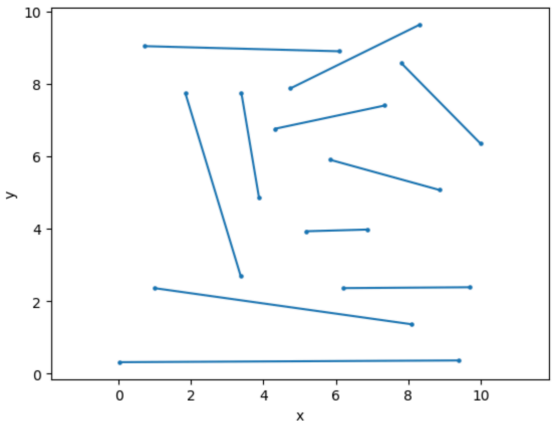
Rysunek 1: Zestaw A



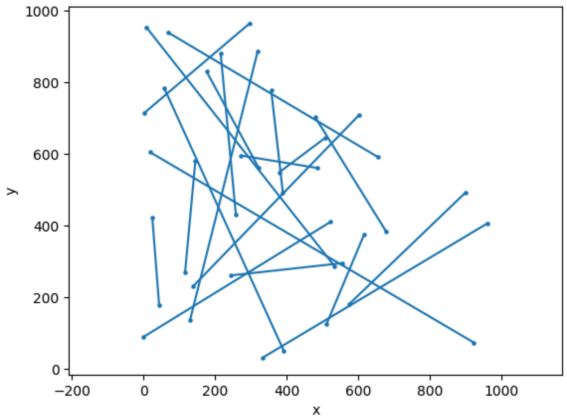
Rysunek 2: zestaw B



Rysunek 3: Zestaw C



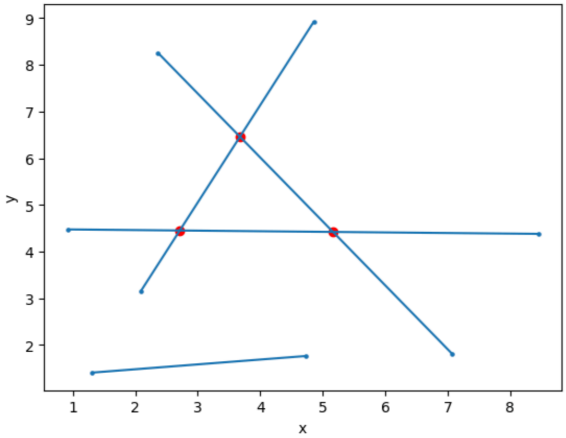
Rysunek 4: Zestaw D



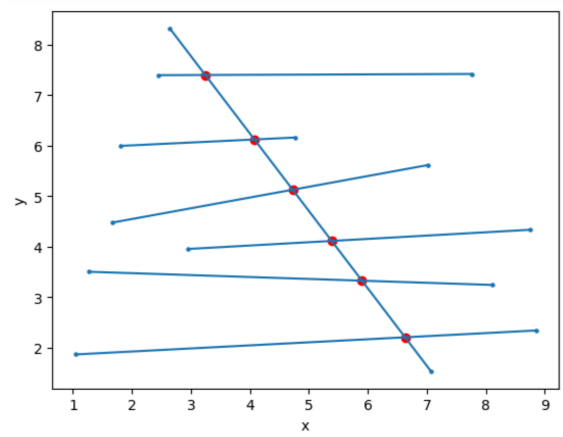
Rysunek 5: Zestaw E

4.2. Analiza wyników

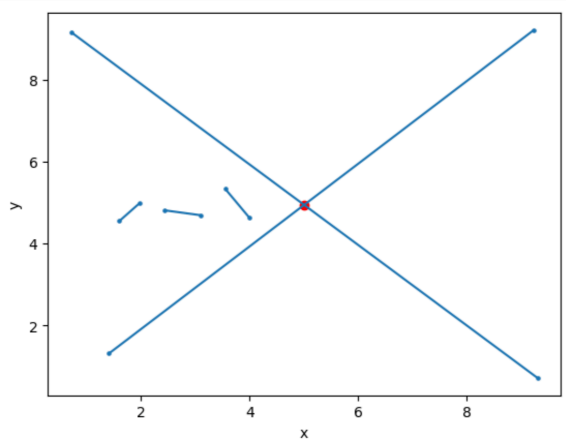
Rysunki 6-10 prezentują rezultaty działania zaimplementowanego algorytmu dla zdefiniowanych wcześniej zestawów danych. Wykryte punkty przecięcia zostały oznaczone na wykresach kolorem czerwonym.



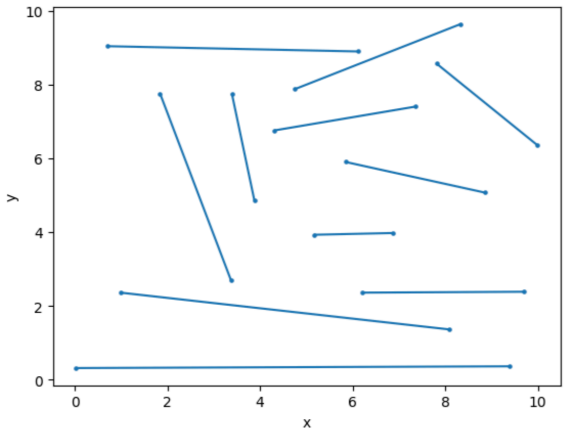
Rysunek 6: Punkty przecięcia dla Zestawu A



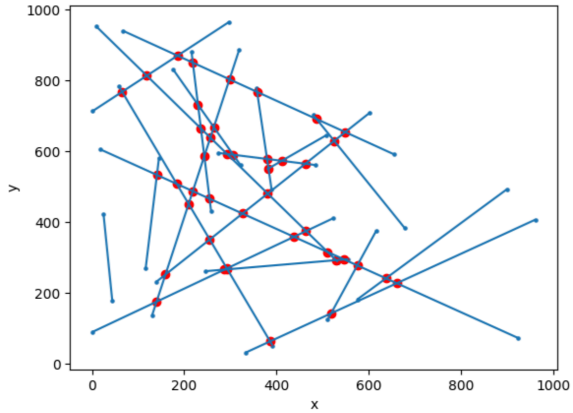
Rysunek 7: Punkty przecięcia dla Zestawu B



Rysunek 8: Punkty przecięcia dla Zestawu C



Rysunek 9: Punkty przecięcia dla Zestawu D



Rysunek 10: Punkty przecięcia dla Zestawu E

Przeprowadzone testy potwierdziły poprawność działania zaimplementowanego rozwiązania dla każdego z analizowanych zestawów danych – zarówno w przypadkach trywialnych, jak i w scenariuszach o wysokim stopniu złożoności.

Dodatkowo, do materiałów laboratoryjnych dołączono wizualizacje działania działania poszczególnych kroków algorytmu w postaci plików GIF.

5. Wnioski

Realizacja ćwiczenia potwierdziła, że algorytm zamiatania jest rozwiązaniem znacznie wydajniejszym od metody naiwnej $O(n^2)$, osiągając złożoność $O((n+k)\log n)$. Kluczowym czynnikiem wpływającym na efektywność było zastosowanie odpowiedniej struktury danych do implementacji struktury stanu `SortedSet`, która umożliwiła utrzymanie porządku odcinków i wyszukiwanie sąsiadów w czasie logarytmicznym. Ostatecznie, przeprowadzone testy oraz wizualizacje krokowe potwierdziły poprawność działania algorytmu dla wszystkich analizowanych scenariuszy, od przypadków trywialnych po gęste sieci losowych odcinków.